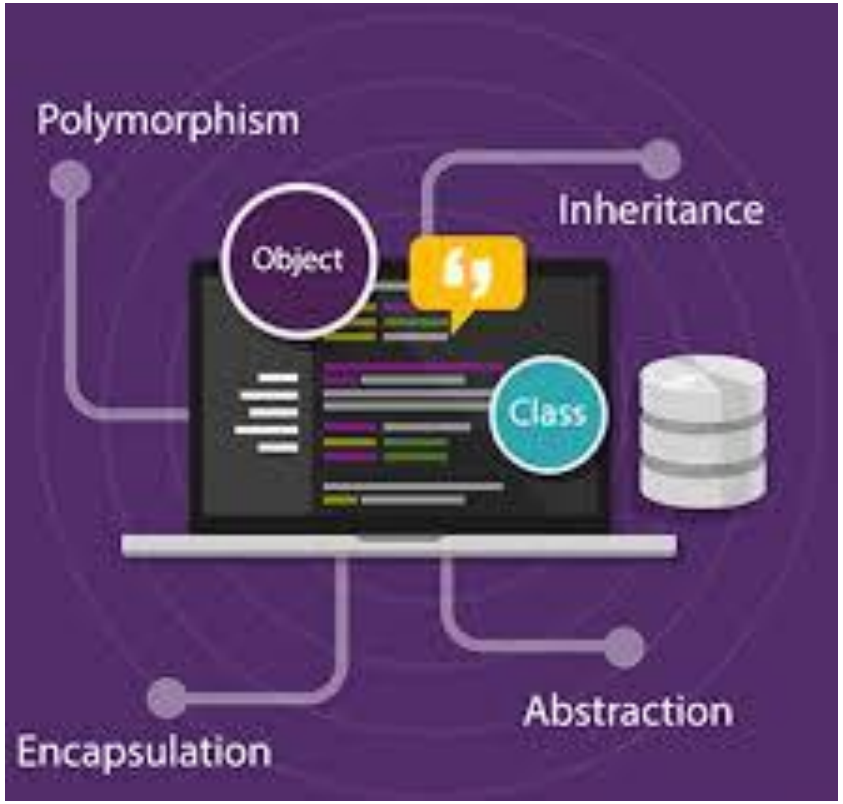
Programmation Orienté Objet



Programmation orienté objet

- Définition de classes permettant d'instancier des Objets avec :
  - Propriétés ~ état
  - Méthodes ~ actions : interagissent avec l'état de l'instance
    - Constructeur(0..n) : appelé une fois à la création d'une instance.
      Permet d'initialiser les propriétés
    - Destructeur (0..1) : appelé lorsque le « garbage collector » libère la mémoire de l'objet
      Utile pour la libération explicite de ressource externe coûteuse

- Une classe peut être vu comme un moule, un gabarit
- Les instances comme les objets résultants

Exercice :

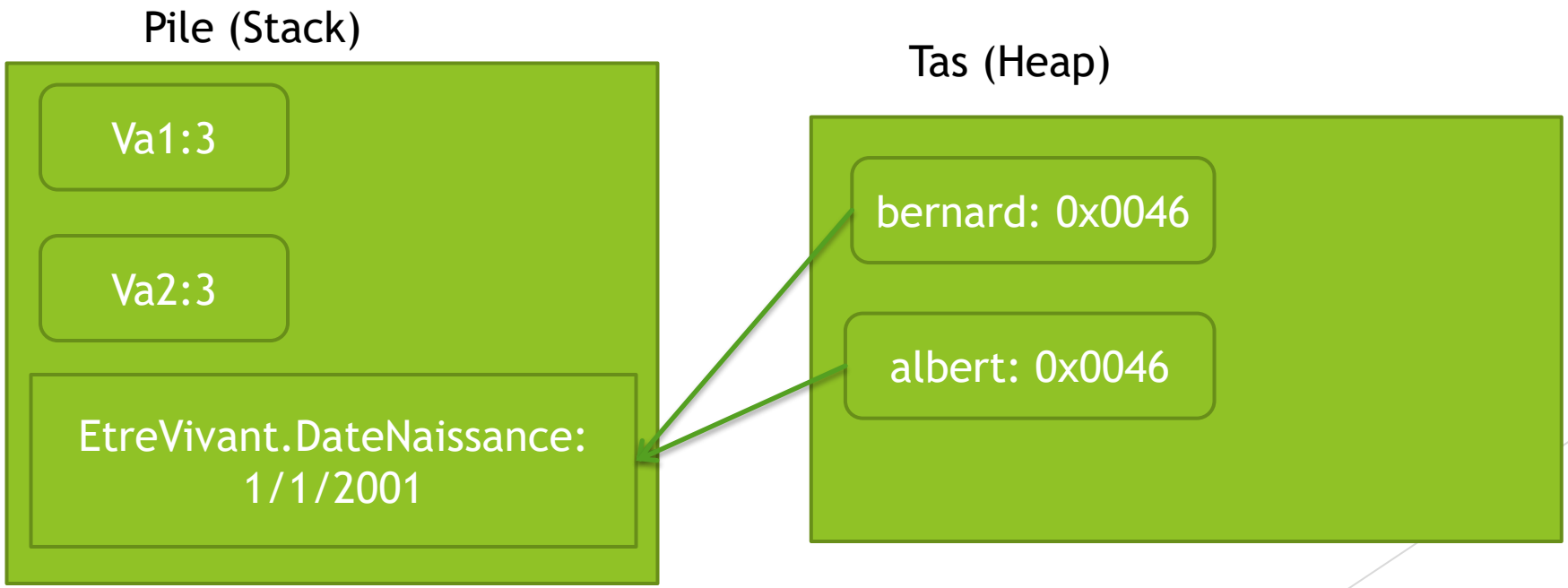
- Créer une classe EtreVivant avec
  - une propriété DateDeNaissance
  - 2 constructeurs
    - Un constructeur avec un paramètre permettant d'initialiser la propriété DateDeNaissance
    - Un constructeur sans paramètre qui initialise la date de naissance à la date courante
  - Une méthode GetAge() qui calcul l'âge de l'être vivant en année pleine

- Instancier 2 être vivants
  - Un éléphant né en 1958
  - Une souris né la semaine dernière

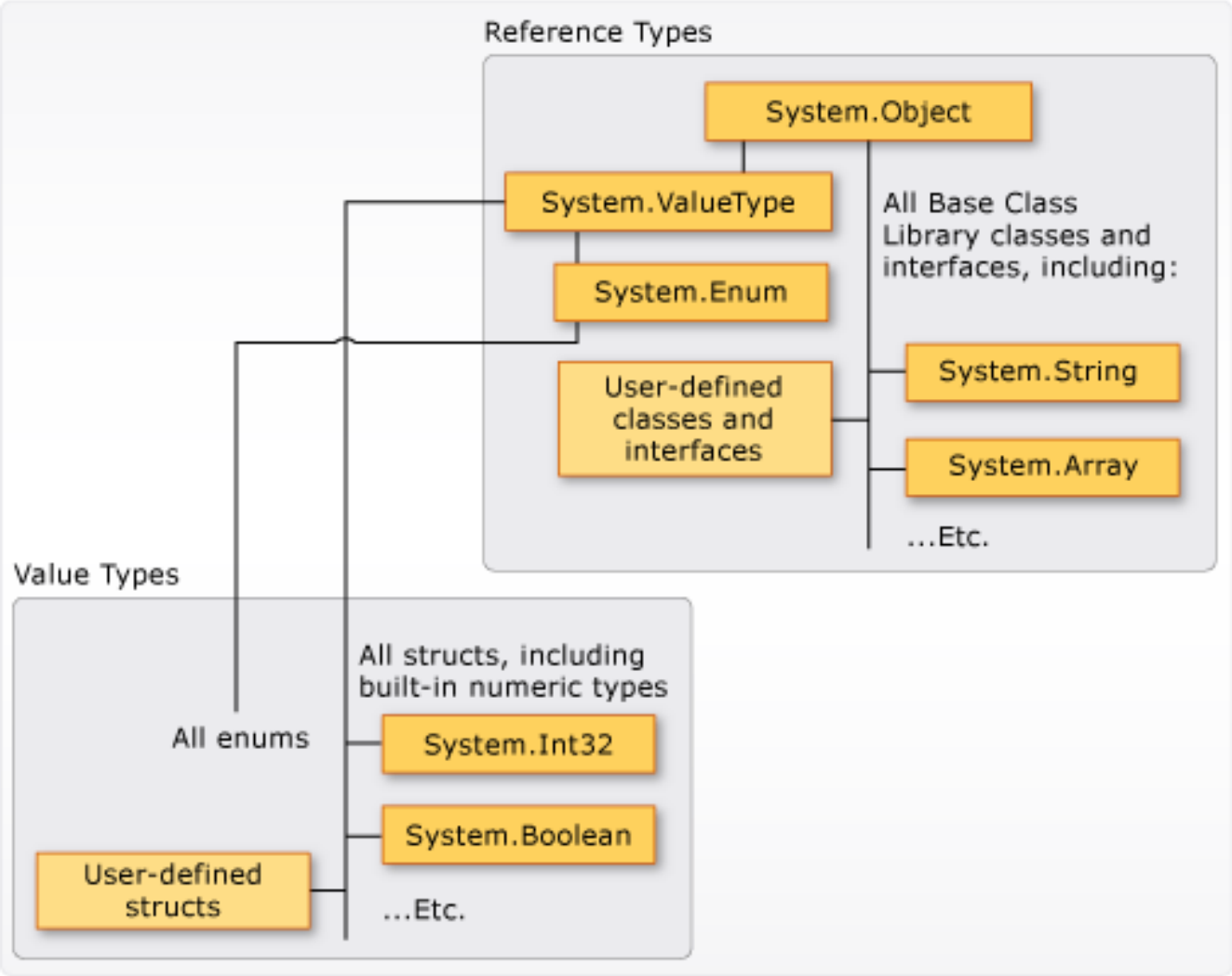
Type valeur / type référence

- Int val1 = 3;
- Int val2 = val1;

- Var bernard = new EtreVivant(1/1/2001);
- Var albert = bernard;



Type valeur / type référence



Exercice

- Déterminez la sortie de la console :

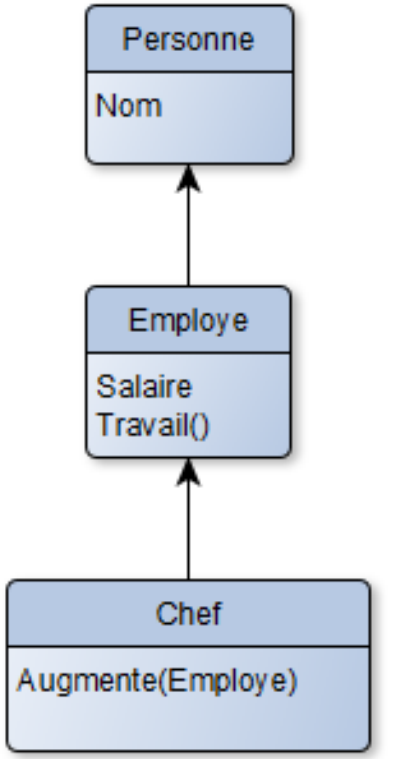
```
1 référence
public static void JeModifieLesParametres(DateTime date, EtreVivant etreVivant)
{
    date = date.AddDays(10);
    Console.WriteLine($"Date : {date:D}");

    etreVivant.DateNaissance = date;
    etreVivant.Nom = $"Jean-{etreVivant.Nom}";
    Console.WriteLine($"Date de naissance de {etreVivant.Nom} : {etreVivant.DateNaissance:D}");
}
```

```
0 références
public static void Main(string[] args)
{
    EtreVivant paul = new EtreVivant(new DateTime(2001, 1, 1))
    {
        Nom = "Paul"
    };
    DateTime date = DateTime.Now.AddYears(-30);

    JeModifieLesParametres(date, paul);

    Console.WriteLine("Ces 2 dates sont-elles identiques ?");
    Console.WriteLine($"   {paul.DateNaissance} : date de naissance de paul");
    Console.WriteLine($"   {date} : date en paramètre ");

}
```

Héritage

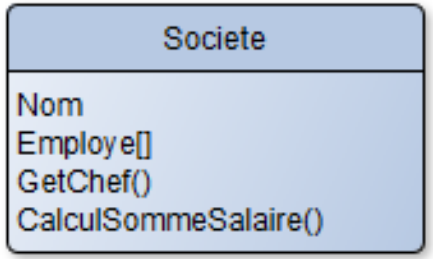
- La dérivation permet de bénéficier de l'implémentation de la classe parente
- Chaque classe / propriété / méthode a un niveau d'accès
  - Private : visible uniquement dans la classe
  - Protected : visible dans la classe et les dérivés
  - Public : visible pour tout le monde
  - Internal : public pour l'assembly uniquement
- Polymorphisme
  - Une classe dérivée peut être manipulée comme si elle était du type de la classe de base
  - Une classe dérivée peut substituer l'implémentation d'une méthode « virtual » avec le mot clef « override »
  - L'accès aux méthodes et propriétés de la classe parente s'effectue avec le mot clé « base »

Exercice : The Office



- Travail() écrit « [Nom] travaille » dans la console
- Le constructeur par défaut prend en paramètre le salaire

- Augmente() : augmente l'employé en paramètre d'une somme fixe
- Travail() écrit « [Nom] zzz $$$ zzz » dans la console



- GetChef(): renvoie le chef de la liste des employés
- Travail() : fait travailler tous les employés

Exercice : The Office
La lutte des classes

- Faire une société avec 5 employés et un chef
- Faites travailler la société
- Exécuter le calcul des salaires
- Le chef augmente le premier salarié
- Le chef s’augmente
- Exécuter à nouveau le calcul des salaires

- Changez le type de retour de GetChef en Employe
  - Que se passe-t-il si on appelle Travail avec la variable résultat ?
  - Peut-on appeler AugmenteEmploye ?

Exercice : The Office
Et la vie privée ?

- Ajouter les propriétés HobbyHonteux à la classe Personne, initialiser les valeurs
- Ajouter une méthode Stalker à Societe qui liste les hobbys honteux de ses employés

- Comment garder son hobby honteux dans l’espace public sans que son employeur ne soit au courant ?

Les interfaces

- Une interface est un **CONTRAT**
  - Membres (propriété et méthodes) à implémenter
  - Une interface ne peut être instanciée !
  - Une interface est destinée à décorer une classe, on dit que la classe implémente l'interface
  - Par convention, le nom d'une interface commence par « I » en majuscule

- Pour aller plus loin
  - Une interface peut hériter d'une interface (mais pas d'une classe !)
  - Une classe peut implémenter plusieurs interfaces

Exercice : The Office A new hope

- Résoudre le problème soulevé au chapitre précédent en créant une interface IEmploye
- Remplacer l'utilisation de la classe Employe par IEmploye pour Societe et Chef

- Question :
  - Societe peut elle trouver les HobbyHonteux de ses employés ?
  - Chef implémente-t-il l'interface IEmploye ?
  - Prendre le 1er employé de société, comment retrouve-t-il son HobbyHonteux ?

Static

- Le mot clé « static » peut décorer une classe, propriété ou des méthodes.

- Une méthode / propriété « static » n’est liée à aucune instance d’objet, elle n’a pas accès aux méthodes / propriétés classiques de la classe, seulement aux autres membres « static »
  L’accès se fait par le nom de la classe directement

- Une classe « static » ne peut pas être instanciée, elle ne peut contenir que des propriétés ou méthodes « static »
  ~Helper

EXCLUSIVITE

Méthodes d'extension

- Méthode statique de classe statique permettant d'étendre les fonctionnalité d'un type dont on a pas le source

- Déclaration

```
namespace ExtensionMethods
{
    public static class MyExtensions
    {
        public static int WordCount(this string str)
        {
            return str.Split(new char[] { ' ', '.', '?' },
                             StringSplitOptions.RemoveEmptyEntries).Length;
        }
    }
}
```

- Utilisation (ne pas oublier le using !)

```
using ExtensionMethods;
```

```
string s = "Hello Extension Methods";
int i = s.WordCount();
```